STEP 1 — Import Libraries

```
# Basic Libraries
import numpy as np
import pandas as pd
import re
import os
import matplotlib.pyplot as plt

# Sklearn
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader

print("Libraries Imported Successfully")
```

```
Libraries Imported Successfully
```

STEP 2 — Load and Explore Dataset

```
import os
import requests
import pandas as pd

file_name = "spam.csv"
url = "https://raw.githubusercontent.com/amankharwal/SMS-Spam-Detection/master/spam.csv" # Updated to a working URL

# Download dataset if not present
if not os.path.exists(file_name) or os.path.getsize(file_name) == 0:
    print("Downloading spam.csv...")
    try:
        response = requests.get(url)
        response.raise_for_status()
        with open(file_name, "wb") as f:
            f.write(response.content)
        print("Download completed successfully.")
    except requests.exceptions.RequestException as e:
        print("Error downloading file:", e)
else:
    print("spam.csv already exists.")

# Load dataset
df = pd.read_csv(file_name, encoding="latin-1")

# Keep required columns (using 'class' and 'message' as observed in the kernel state)
df = df[['class', 'message']]
df.columns = ['label', 'text']

# Encode labels
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

# Display dataset information
print(df.head())
print("\nTotal Samples:", len(df))
print("\nClass Distribution:\n", df['label'].value_counts())
```

```
spam.csv already exists.
   label                                               text
0      0  Go until jurong point, crazy.. Available only ...
1      0                      Ok lar... Joking wif u oni...
2      1  Free entry in 2 a wkly comp to win FA Cup fina...
3      0  U dun say so early hor... U c already then say...
4      0  Nah I don't think he goes to usf, he lives aro...

Total Samples: 5572

Class Distribution:
 label
0    4825
1     747
Name: count, dtype: int64
```

STEP 3 — Text Preprocessing

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    return text

df['text'] = df['text'].apply(clean_text)

# Tokenization
df['tokens'] = df['text'].apply(lambda x: x.split())

df.head()
```

	label	text	tokens
0	0	go until jurong point crazy available only in ...	[go, until, jurong, point, crazy, available, o...
1	0	ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]
2	1	free entry in a wkly comp to win fa cup final...	[free, entry, in, a, wkly, comp, to, win, fa, ...
3	0	u dun say so early hor u c already then say	[u, dun, say, so, early, hor, u, c, already, t...
4	0	nah i dont think he goes to usf he lives aroun...	[nah, i, dont, think, he, goes, to, usf, he, l...

Next steps: Generate code with df New interactive sheet

STEP 4 — Vocabulary and Pretrained Embedding Preparation.. Create Vocabulary

```
vocab = {}
for tokens in df['tokens']:
    for word in tokens:
        if word not in vocab:
            vocab[word] = len(vocab) + 2  # reserve 0,1

vocab['<PAD>'] = 0
vocab['<UNK>'] = 1

vocab_size = len(vocab)
print("Vocabulary Size:", vocab_size)
```

```
Vocabulary Size: 8540
```

Load GloVe Embeddings (Download glove.6B.100d.txt first)

```
EMBEDDING_DIM = 100

# Download glove.6B.100d.txt if it doesn't exist
import os
if not os.path.exists('glove.6B.100d.txt'):
    print("Downloading glove.6B.100d.txt...")
    !wget -q http://nlp.stanford.edu/data/glove.6B.zip
    !unzip -q glove.6B.zip
    !rm glove.6B.zip
    print("Download and extraction completed.")

embedding_index = {}
with open("glove.6B.100d.txt", encoding="utf8") as f:
    for line in f:
        values = line.split()
        word = values[0]
        vector = np.asarray(values[1:], dtype='float32')
        embedding_index[word] = vector

print("Loaded GloVe vectors:", len(embedding_index))
```

```
Downloading glove.6B.100d.txt...
Download and extraction completed.
Loaded GloVe vectors: 400000
```

Create Embedding Matrix

```
embedding_matrix = np.zeros((vocab_size, EMBEDDING_DIM))

for word, idx in vocab.items():
```

```
    vector = embedding_index.get(word)
    if vector is not None:
        embedding_matrix[idx] = vector
    else:
        embedding_matrix[idx] = np.random.normal(scale=0.6, size=(EMBEDDING_DIM,))
```

STEP 5 — Train-Test Split

```
X = df['tokens']
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Convert Text to Numerical Sequences

```
MAX_LEN = max([len(tokens) for tokens in df['tokens']])

def encode_text(tokens):
    encoded = [vocab.get(word, vocab['<UNK>']) for word in tokens]

    if len(encoded) < MAX_LEN:
        encoded += [vocab['<PAD>']] * (MAX_LEN - len(encoded))
    else:
        encoded = encoded[:MAX_LEN]

    return encoded

X_train_enc = [encode_text(tokens) for tokens in X_train]
X_test_enc = [encode_text(tokens) for tokens in X_test]
```

Create PyTorch Dataset

```
class TextDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.long)
        self.y = torch.tensor(y.values, dtype=torch.float32)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

train_dataset = TextDataset(X_train_enc, y_train)
test_dataset = TextDataset(X_test_enc, y_test)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32)
```

STEP 6 — Build 1D CNN Model with Pretrained Embeddings

```
class TextCNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, embedding_matrix):
        super(TextCNN, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.embedding.weight = nn.Parameter(
            torch.tensor(embedding_matrix, dtype=torch.float32)
        )

        self.conv = nn.Conv1d(in_channels=embedding_dim,
                              out_channels=128,
                              kernel_size=5)

        self.pool = nn.MaxPool1d(kernel_size=2)

        # Correctly calculate the input features for the FC layer
        # Based on MAX_LEN = 171, kernel_size=5 (conv), kernel_size=2 (pool)
        # Conv output length: (171 - 5) + 1 = 167
        # Pool output length: floor(167 / 2) = 83
        # Flattened size: 128 * 83 = 10624
```

```
        self.fc = nn.Linear(128 * 83, 1)  # Corrected calculation

        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.embedding(x)
        x = x.permute(0, 2, 1)
        x = self.conv(x)
        x = self.pool(x)
        x = torch.flatten(x, 1)
        x = self.fc(x)
        return self.sigmoid(x)

model = TextCNN(vocab_size, EMBEDDING_DIM, embedding_matrix)
print(model)
```

```
TextCNN(
  (embedding): Embedding(8540, 100)
  (conv): Conv1d(100, 128, kernel_size=(5,), stride=(1,))
  (pool): MaxPool1d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc): Linear(in_features=10624, out_features=1, bias=True)
  (sigmoid): Sigmoid()
)
```

STEP 7 — Model Training

```
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

EPOCHS = 5

for epoch in range(EPOCHS):
    model.train()
    total_loss = 0

    for inputs, labels in train_loader:
        optimizer.zero_grad()

        outputs = model(inputs).squeeze()
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss/len(train_loader):.4f}")
```

```
Epoch 1, Loss: 0.2386
Epoch 2, Loss: 0.0674
Epoch 3, Loss: 0.0307
Epoch 4, Loss: 0.0181
Epoch 5, Loss: 0.0098
```

STEP 8 — Model Evaluation

```
model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for inputs, labels in test_loader:
        outputs = model(inputs).squeeze()
        preds = (outputs >= 0.5).float()

        all_preds.extend(preds.numpy())
        all_labels.extend(labels.numpy())

accuracy = accuracy_score(all_labels, all_preds)
precision = precision_score(all_labels, all_preds)
recall = recall_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds)
cm = confusion_matrix(all_labels, all_preds)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("Confusion Matrix:\n", cm)
```

```
Accuracy: 0.9811659192825112
Precision: 0.951048951048951
Recall: 0.9066666666666666
F1 Score: 0.9283276450511946
Confusion Matrix:
 [[958   7]
 [ 14 136]]
```